

Notification Framework

User Functional Document

1 Introduction

1.1 Purpose

This document describes the functional behaviour of the Notification Framework — a backend service that accepts notification requests from client applications and reliably delivers messages across Email, SMS, WhatsApp, and Push channels. It is intended for developers integrating with the API, operations engineers monitoring the service, and QA teams validating end-to-end delivery.

1.2 Scope

The Notification Framework encompasses:

- A FastAPI HTTP layer that exposes REST endpoints for notification dispatch, preference management, provider administration, queue control, media upload, and analytics.
- A Celery-based asynchronous worker fleet that consumes jobs from RabbitMQ and calls third-party providers (SendGrid, Twilio, UltraMsg, Firebase FCM).
- A MongoDB persistence layer storing notification requests, jobs, delivery logs, dead-letter queue entries, and client configuration.
- A Dead Letter Queue (DLQ) processor that retries or permanently drops failed jobs according to configurable policies.
- A queue-failover monitor that automatically switches to a backup queue when the primary queue becomes congested or stalled.

1.3 Definitions & Abbreviations

Term	Meaning
DLQ	Dead Letter Queue — storage for jobs that exhausted all delivery attempts
FCM	Firebase Cloud Messaging — Google's push notification platform
IST	India Standard Time (UTC+5:30) — used for all timestamps in logs
job_id	Unique identifier for a single delivery attempt unit
notification_id	Identifier grouping all jobs for a single notification request
request_id	Identifier for the top-level API call from a client
provider_index	Zero-based index into providers_snapshot selecting the active provider
providers_snapshot	Ordered list of provider configurations frozen at dispatch time
QUEUED / PROCESSING / SENT / FAILED / DLQ / DROPPED	Canonical job lifecycle states

Term	Meaning
D	
channel	Delivery modality: email sms whatsapp push
rate_limit	Maximum jobs per minute allowed through a channel queue
countdown	Seconds Celery waits before re-attempting a retried task

1.4 System Overview

Clients authenticate via Bearer token and POST a notification request to /notify. The API validates the payload, resolves recipient preferences, snapshots provider configuration, creates one notification_job per recipient, and enqueues each job on the appropriate RabbitMQ queue (email-notify-q, sms-notify-q, whatsapp-notify-q, push-notify-q).

A Celery worker picks up each job, checks channel-level pause/rate-limit controls, selects the active provider from the snapshot, and calls the provider SDK. On success a delivery_log entry is written with status SENT. On transient failure the worker retries the same provider (exponential back-off) or switches to the next provider in the snapshot. When all providers are exhausted the job is moved to the DLQ.

A background DLQ processor polls the dlq collection every 5 seconds and re-enqueues recoverable jobs (with appropriate delay or provider rotation) while permanently dropping non-recoverable ones. A failover monitor thread watches primary queue depth and switches the active queue pointer for a channel when congestion is detected.

2 Architecture & Component Map

2.1 High-Level Components

Component	Technology	Responsibility
FastAPI App (main.py)	Python / FastAPI / Uvicorn	HTTP ingress, request validation, job dispatch, webhook reception, analytics
Celery Workers (tasks.py)	Celery 5 / Python	Async job execution, provider dispatch, delivery logging, retry orchestration
RabbitMQ	RabbitMQ 3 (AMQP)	Message broker with four channel-specific queues and optional backup queues
MongoDB	MongoDB 7	Persistent store for all collections (requests, jobs, logs, DLQ, providers, clients)
DLQ Processor (dlq_processor.py)	Python daemon thread	Polls DLQ, re-enqueues or drops jobs based on error classification
Queue Failover Monitor	Python daemon thread inside FastAPI	Watches primary queue depth, switches active_queue pointer on congestion
Celery Config (celery_config.py)	Celery / Kombu	Queue definitions, serialiser settings, ack policy, prefetch multiplier

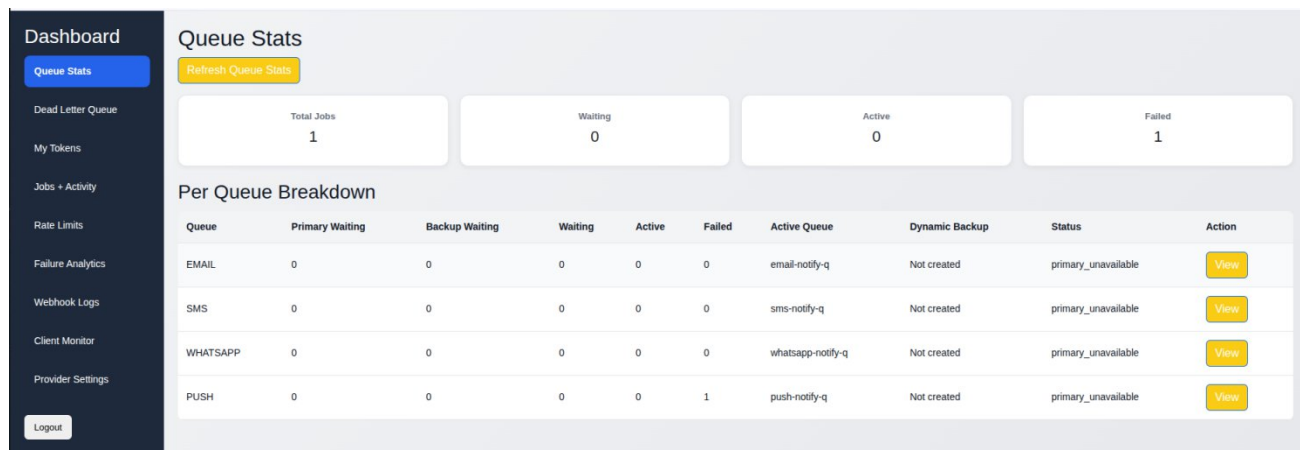
2.2 MongoDB Collections

Collection	Primary Key	Description
notification_requests	request_id	Top-level API call record; one per /notify invocation
notification_jobs	job_id	One per recipient per channel; tracks lifecycle state
delivery_logs	log_id	Immutable delivery outcome written by the worker on SENT or FAILED
dlq	job_id	Dead-letter entries with error classification and requeue meta data
providers	provider_id	Provider credentials and configuration per channel
clients	client_id	Client API keys, webhook_url, and channel bindings
user_preferences	user_id + channel	Per-user opt-in/opt-out and preferred channel overrides
queue_controls	channel	Runtime pause, rate_limit, active_queue, and backup_queue state
queue_rate_counters	channel + window	Rolling counter for per-minute rate enforcement
rate_limit_logs	—	Per-recipient rate limit tracking (client-level)
global_rate_limit_logs	—	System-level rate limit tracking
webhook_calls	call_id	Audit log for outbound webhook fan-out to clients
media_files	file_id	Uploaded attachment metadata, access tokens, and TTL
ultramsg_inbound	—	Raw inbound WhatsApp webhook payloads for debugging

2.3 RabbitMQ Queue Layout

Queue Name	Channel	Notes
email-notify-q	Email	Primary; backup queue declared dynamically via task_create_missing_queues
sms-notify-q	SMS	Primary
whatsapp-notify-q	WhatsApp	Primary
push-notify-q	Push / FCM	Primary
<channel>-notify-q-backup	Any	Backup queue; failover monitor switches active_queue pointer here on congestion

Monitoring of each queue in admin dashboard:



3 Functional Documentation

3.1 Sending a Notification — POST /notify

Purpose. Dispatch one notification across one or more channels to one or more recipients.

1. Client sends POST /notify with JSON body containing: event_type, channel, recipients list (each with user_id and address fields), content (subject, body, attachments), and optional retry_policy.
2. API validates the payload (pydantic) and resolves per-user preferences from user_preferences collection. Opted-out recipients are skipped and logged.
3. Client-level rate limit is checked against rate_limit_logs. If exceeded, HTTP 429 is returned.
1. Provider snapshot is fetched from providers collection ordered by priority; frozen into providers_snapshot on the job.
2. A notification_request document is written; one notification_job per (recipient × channel) is written with status QUEUED.
3. Each job is enqueued to the active queue for its channel via Celery apply_async(queue=active_queue).
4. API returns HTTP 200 with a list of job_ids and their initial statuses.

Outcome. Jobs are persisted and queued; delivery is asynchronous.

Edge cases / user actions.

- No providers configured for channel → job created with status FAILED; error returned in response.
- Global rate limit exceeded → HTTP 429.
- Recipient has opted out → job skipped; status SKIPPED recorded in response.
- Content exceeds size limits per channel → HTTP 422 with violations payload.

Test screen for /notify: <http://localhost:5174/>

Notify via JSON Valid JSON

Edit the payload and send directly to the API

X-API-Key
Sent on every request - stored only in this browser

..... Show

Media Upload Local file Remote URL

Upload once to get a `file_id` for `content.attachments`

CHOOSE FILE

Select file No file selected

NAME **MIME TYPE**

Optional display name application/pdf

SIZE BYTES **DELIVERY MODE**

Optional auto

Upload media Reset fields

`payload.json` 18 lines Copy Format Reset

```

1 {
2   "client_id": "client_001",
3   "event_type": "PAYMENT_SUCCESS",
4   "recipients": [
5     {
6       "user_id": "user_1",
7       "email": "test@gmail.com",
8       "phone": "9876543210",
9       "wa_number": "9876543210",
10      "fcm_token": "token"
11    }
12  ],
13  "channels_requested": ["email"],
14  "priority": "HIGH",
15  "content": {
16    "message": "Payment successful!"
17  }
18 }
```

Send notification

3.2 Worker Job Execution

Purpose. Asynchronous delivery of a notification job by the Celery worker.

1. Worker picks up job from its queue (email-notify-q etc.).
2. Worker checks `queue_controls` for the channel: if `paused=true`, it re-enqueues with a 5-second countdown and returns.
3. Worker enforces `rate_limit` (jobs/min) via `queue_rate_counters`. If the per-minute budget is exhausted, it retries with a calculated countdown.
4. Worker selects `providers_snapshot[provider_index]` and calls the appropriate provider (SMTP, Twilio, UltraMsg, FCM).
5. On **SUCCESS**: `delivery_log` is written (`status=SENT`); `notification_job` status set to `SENT`; client webhook emitted.
6. On **RETRY_SAME_PROVIDER**: exponential back-off ($10s \times 2^{\text{attempt}}$, max 300s); job re-enqueued.
7. On **SWITCH_PROVIDER**: `provider_index` incremented; job re-enqueued with 2-second countdown.
8. On **ALL_FAILED** (all providers exhausted): `delivery_log` written (`status=FAILED`); job pushed to DLQ; `notification_job` status set to DLQ.

Outcome. Successful delivery produces a `SENT` log; all-provider failure produces a DLQ entry.

3.3 Provider Failerover

Purpose. Maximise delivery success by cascading across multiple configured providers.

`providers_snapshot` is an ordered list of provider configurations frozen at dispatch time. `provider_index` tracks the currently active provider. The `dispatch_with_failover()` function tries providers in order and advances the index on failure. Provider history is appended to `provider_history` on each attempt.

Dispatch Result	Action	Behaviour
Provider call succeeds	SUCCESS	Log SENT, emit webhook, return
Transient error (timeout, 5xx)	RETRY_SAME_PROVIDER	Exponential back-off up to 5 minutes; up to <code>max_retries</code> per provider
Provider hard-failure (auth, invalid)	SWITCH_PROVIDER	Increment <code>provider_index</code> , re-enqueue immediately
All providers tried	ALL_FAILED	Push to DLQ, log FAILED

3.4 Dead Letter Queue (DLQ) Processing

Purpose. Recover or discard jobs that could not be delivered by the primary worker.

The DLQ processor runs as a daemon thread polling dlq every 5 seconds. For each job with `status=DLQ` and `next_retry_at ≤ now`:

- If `requeue_count ≥ 3` → DROPPED (max attempts reached).
- If `error_type = PERMANENT` (invalid address, unsubscribed, auth failed) → DROPPED immediately.
- If `error_code = RATE_LIMIT_EXCEEDED` → re-enqueued with delay derived from `next_retry_at` (default 30 minutes).
- If `error_code = SMTP_TIMEOUT` → re-enqueued with provider switch (`provider_index + 1`).
- All others → re-enqueued normally.

On re-enqueue: DLQ entry `status` → `REQUEUED`, `requeue_count` incremented; `notification_job` status reset to `QUEUED` on the backup or active queue.

Edge cases.

- Original job not found in `notification_jobs` → DROPPED.
- No recipient address resolvable for channel → DROPPED.
- `providers_snapshot` empty → DROPPED.

3.5 Queue Failover Monitor

Purpose. Automatically reroute traffic to a backup queue when the primary queue becomes congested or stuck.

A background thread polls each channel queue every `QUEUE_FAILOVER_MONITOR_INTERVAL_SEC` seconds (default 5). Conditions triggering failover:

- Queue depth > `PRIMARY_QUEUE_CONGESTION_DEPTH` (default 100 messages).
- Queue depth is not decreasing for `PRIMARY_QUEUE_STUCK_SECONDS` (default 60 seconds).

On congestion: `active_queue` in `queue_controls` is set to the backup queue name. Subsequent /notify calls enqueue to the backup queue. Recovery: when backup queue depth falls below `PRIMARY_QUEUE_RECOVER_DEPTH` the monitor switches `active_queue` back to the primary.

Configuration (environment variables).

Variable	Default	Meaning
QUEUE_FAILOVER_ENABLED	1	Enable/disable the failover monitor
PRIMARY_QUEUE_CONGESTION_DEPTH	100	Messages threshold to trigger failover
PRIMARY_QUEUE_MAX_BUFFER	500	Maximum queue depth before hard rejection
PRIMARY_QUEUE_RECOVER_DEPTH	50	Backup queue depth threshold to revert to primary
PRIMARY_QUEUE_STUCK_SECONDS	60	Inactivity window to declare queue stuck
QUEUE_FAILOVER_MONITOR_INTERVAL_SEC	5	Polling interval in seconds
QUEUE_FAILOVER_MIN_SWITCH_SECONDS	30	Minimum seconds between consecutive switch events

3.6 Queue Controls — Pause & Rate Limit

Purpose. Allow operators to pause a channel or throttle its throughput without restarting workers.

Endpoint	Method	Action
/queue-controls/{channel}	GET	Fetch current pause and rate_limit state for a channel
/queue-controls/{channel}/pause	POST	Pause the channel queue (workers re-enqueue with 5s countdown)
/queue-controls/{channel}/resume	POST	Resume a paused channel queue
/queue-controls/{channel}/rate-limit	POST	Set jobs-per-minute rate limit (0 = unlimited)

Rate limiting uses a sliding-window counter in queue_rate_counters (per channel, per minute bucket). When the budget is exhausted the worker calls self.retry(countdown=N) where N is the remaining seconds in the current window.

On hitting view button on Queue stats page:

Queue Control — EMAIL

Close

Refresh Queue Status

Failover Status

Primary Queue

email-notify-q

Dynamic Backup Queue

Not created

Primary Waiting

0

Last Failover Reason

None

Active Queue

email-notify-q

Current State

primary_unavailable

Backup Waiting

0

Last Failover Time

-

Pause

Resume

Clear Queue

Retry Failed

Reprocess DLQ

Rate Limit (jobs/min)

0

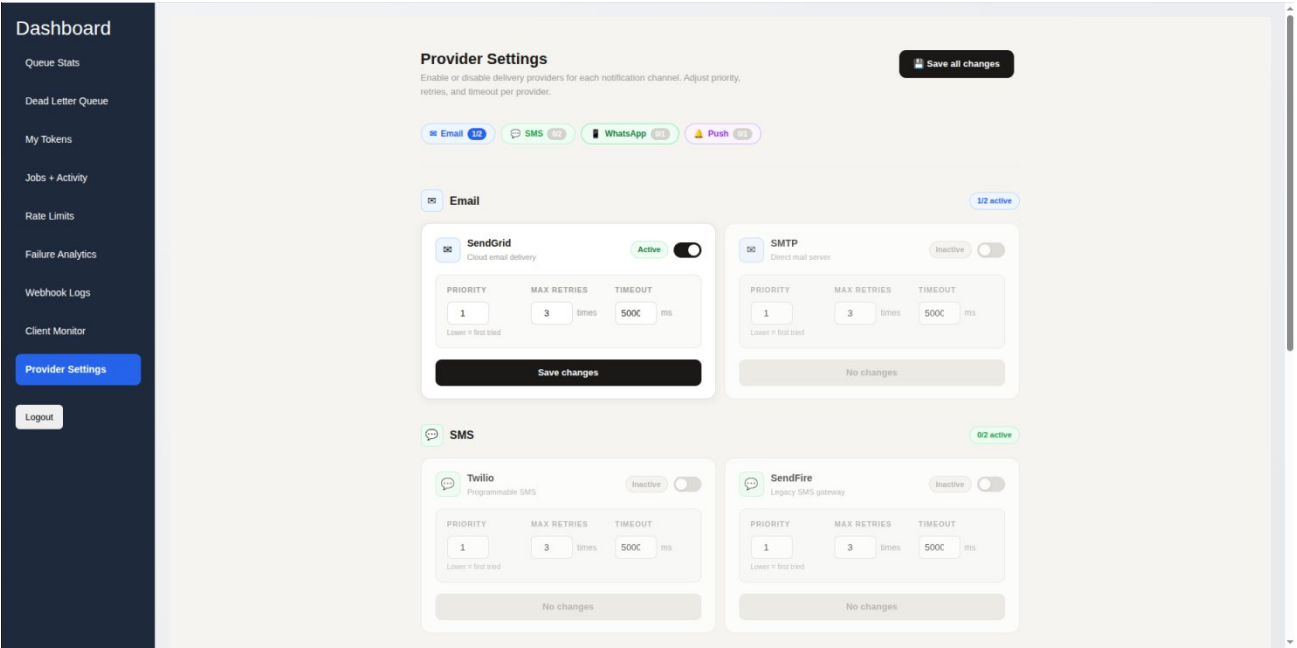
Update

3.7 Provider Management

Purpose. CRUD operations for provider configurations used by the worker.

Endpoint	Method	Description
/providers	GET	List all providers (credentials masked)
/providers	POST	Create a new provider for a channel
/providers/{provider_id}	PUT	Update provider credentials or priority
/providers/{provider_id}	DELETE	Remove a provider
/providers/{provider_id}/test	POST	Send a test message via the provider

Each provider document carries: provider_id, provider_name (smtp | twilio | ultramsg | fcm), channel, priority (integer, lower = higher priority), and channel-specific credential fields.

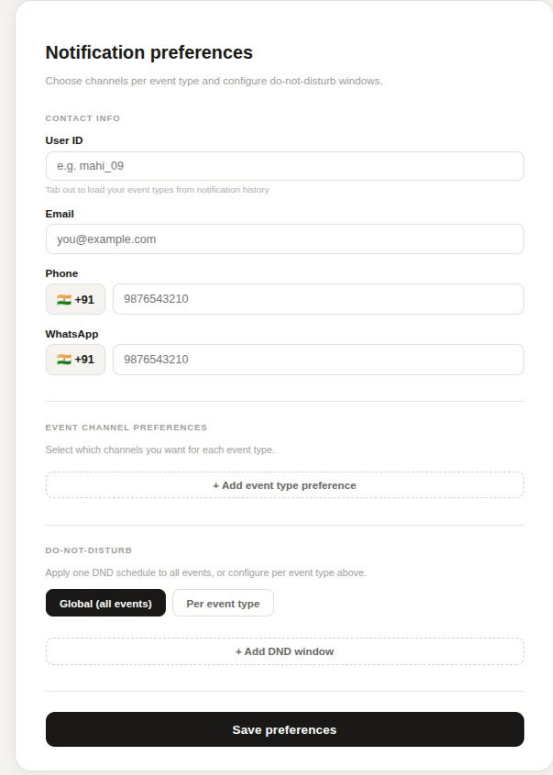


3.8 User Preferences

Purpose. Store and enforce per-user opt-in/opt-out decisions per channel.

Endpoint	Method	Description
/preferences/{user_id}	GET	Retrieve all channel preferences for a user
/preferences/{user_id}	POST/PUT	Upsert preferences (opted_in boolean per channel)
/preferences/{user_id}/{channel}	DELETE	Remove a specific channel preference (reverts to default opted-in)

During /notify dispatch, the API queries user_preferences for each recipient. Recipients with opted_in=false for the requested channel are skipped; the response includes SKIPPED entries with reason 'opted_out'.



Notification preferences

Choose channels per event type and configure do-not-disturb windows.

CONTACT INFO

User ID

Tab out to load your event types from notification history

Email

Phone

WhatsApp

EVENT CHANNEL PREFERENCES

Select which channels you want for each event type.

DO-NOT-DISTURB

Apply one DND schedule to all events, or configure per event type above.

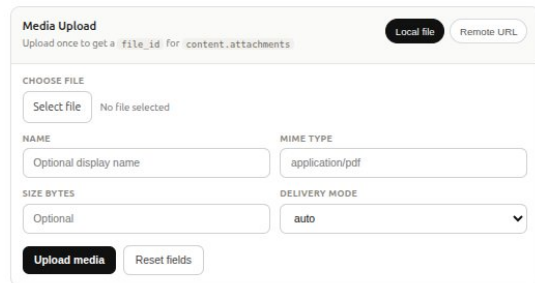
3.9 Media Upload & Attachment

Purpose. Allow clients to upload attachment files that are referenced by `file_id` in notification content.

5. Client POSTs multipart file to `/media/upload`. Optionally provides `delivery_mode` (inline | attachment) and `ttl_days`.
6. Server stores the file in `MEDIA_DIR` and creates a `media_files` document with a time-limited `access_token`.
7. Server returns `file_id` and a presigned URL valid for `MEDIA_URL_TTL_DAYS` days.
8. Client includes `{"file_id": "<id>"}` in `content.attachments` when calling `/notify`.
9. At dispatch time `_normalize_attachments()` resolves `file_id` to URL, refreshing expired tokens automatically.
10. Worker calls `_refresh_content_attachment_urls()` before provider dispatch to ensure URLs remain valid across DLQ retries.

Edge cases.

- `source_url` (externally hosted) — passed through without token; reachability is checked via `_is_publicly_reachable_url`.
- Private/loopback URLs are rejected unless `ALLOW_PRIVATE_MEDIA_URLS=1` or the host is in `ALLOWED_PRIVATE_MEDIA_HOSTS`.



Media Upload
Upload once to get a `file_id` for `content.attachments`

Local file Remote URL

CHOOSE FILE
Select file No file selected

NAME
Optional display name

MIME TYPE
application/pdf

SIZE BYTES
Optional

DELIVERY MODE
auto

Upload media Reset fields

3.10 Delivery Status & Webhooks

Purpose. Provide real-time and queryable delivery status to client applications.

3.10.1 Status Polling

Endpoint	Description
/jobs/{job_id}	Fetch status and metadata for a single job
/jobs	List jobs with filters (channel, status, date range, recipient)
/delivery-logs/{job_id}	Fetch the delivery_log for a job
/failure-analytics	Aggregated failure/delivery rates by event_type

3.10.2 Outbound Client Webhooks

On each SENT or status-update event the worker calls `_emit_client_webhook()`, which POSTs a JSON payload to the client's configured `webhook_url` (5-second timeout, best-effort, never raises). The payload schema:

Field	Type	Description
type	string	Always "NOTIFICATION_STATUS"
job_id	string	Job identifier
request_id	string	Original request identifier
client_id	string	Client identifier
event_type	string	Application event that triggered the notification
channel	string	email sms whatsapp push
status	string	SENT DELIVERED READ FAILED
provider	string	Provider name that handled delivery
provider_message_id	string	Provider-assigned message identifier
timestamp	ISO 8601 string	Event time in IST

3.10.3 Inbound Provider Webhooks

Endpoint	Provider	Events handled
/webhooks/sendgrid	SendGrid (Email)	DELIVERED, OPENED, BOUNCED, FAIL

Endpoint	Provider	Events handled
		ED status updates
/webhooks/twilio	Twilio (SMS)	DELIVERED, UNDELIVERED, FAILED status updates
/webhooks/ultrams	UltraMsg (WhatsApp)	sent, delivered, read, failed status updates; supports batch payloads
/webhooks/ultrams/latest	UltraMsg (debug)	Returns last inbound payload (Bearer-protected)

On receipt of a provider webhook the API finds the matching delivery_log by provider_message_id, job_id, or recipient address, updates its status (and delivered_at/read_at timestamps), updates the parent notification_job, and fans out a client webhook.

3.11 Client Management

Purpose. Register and manage API clients.

Endpoint	Method	Description
/clients	GET	List all clients
/clients	POST	Create a new client; returns api_key
/clients/{client_id}	GET	Fetch a single client record
/clients/{client_id}	PUT	Update client fields (webhook_url, rate limits, etc.)
/clients/{client_id}	DELETE	Deactivate a client

Dashboard Queue Stats Dead Letter Queue My Tokens Jobs + Activity Rate Limits	Client Monitor Search by client id or name All Select Delete All Accounts						
	Client ID	Name	Status	Monthly Quota	Allowed Channels	Created At	Actions
	f38783586b68f386138bd52ddaf85a2	IITH	Active	10000000 Save	email, sms, whatsapp, push	5/5/2026, 4:30:35 PM	Deactivate Rotate Key Delete Account
	929da9d987ec91e8630a30f00f6cf7c	IITH	Active	10000000 Save	email, sms, whatsapp, push	5/4/2026, 10:48:53 PM	Deactivate Rotate Key Delete Account

4 API Reference Summary

4.1 Notification Endpoints

Endpoint	Method	Auth	Description
/notify	POST	Bearer	Dispatch a notification request
/notify/batch	POST	Bearer	Dispatch multiple notification requests in one call
/jobs	GET	Bearer	List notification jobs with filters
/jobs/{job_id}	GET	Bearer	Get job status and metadata

Endpoint	Method	Auth	Description
/delivery-logs/{job_id}	GET	Bearer	Get delivery log for a job
/failure-analytics	GET	Bearer	Get failure/delivery rate analytics by event_type

4.2 Management Endpoints

Endpoint	Method	Auth	Description
/providers	GET/POST	Bearer	List or create providers
/providers/{id}	PUT/DELETE	Bearer	Update or remove a provider
/providers/{id}/test	POST	Bearer	Send test message via provider
/clients	GET/POST	Bearer	List or create clients
/clients/{id}	GET/PUT/DELETE	Bearer	Manage a single client
/preferences/{user_id}	GET/POST	Bearer	Read or update user preferences
/queue-controls/{channel}	GET	Bearer	Read queue control state
/queue-controls/{channel}/pause	POST	Bearer	Pause a channel
/queue-controls/{channel}/resume	POST	Bearer	Resume a paused channel
/queue-controls/{channel}/rate-limit	POST	Bearer	Set jobs/min throttle
/media/upload	POST	Bearer	Upload attachment file
/media/{file_id}	GET	Token	Download attachment (presigned URL)

4.3 Webhook Endpoints

Endpoint	Method	Auth	Description
/webhooks/sendgrid	POST	Signature	Receive SendGrid delivery events
/webhooks/twilio	POST	Signature	Receive Twilio delivery events
/webhooks/ultramsg	POST	None / token	Receive UltraMsg WhatsApp events
/webhooks/ultramsg/latest	GET	Bearer	Debug: last UltraMsg payload

5 Job Lifecycle

5.1 State Diagram

Each notification_job transitions through the following states:

State	Set by	Meaning
QUEUED	API /notify	Job written to DB and enqueued on RabbitMQ
PROCESSING	Worker (start)	Worker has dequeued and started executing the job
SENT	Worker (success)	Provider accepted the message; delivery_log written
FAILED	Worker (all failed)	All providers failed; delivery_log written with error
DLQ	Worker (all failed)	Job entry written to dlq collection for recovery processing
REQUEUED	DLQ processor	DLQ processor re-enqueued the job; notification_job reset to QUEUED
DROPPED	DLQ processor	Job permanently discarded (max retries or permanent error)
SKIPPED	API /notify	Recipient opted out; no job created

5.2 Delivery Log Status

delivery_log.status tracks the downstream delivery outcome updated by provider webhooks:

Status	Meaning
SENT	Worker confirmed provider accepted the message
DELIVERED	Provider confirmed recipient device received the message
READ	Recipient opened/read the message (if supported by provider)
FAILED	Provider reported final delivery failure

6 Channels & Provider Integrations

6.1 Email

Attribute	Detail
Supported providers	Any SMTP server (Gmail, SendGrid SMTP, custom)
Queue	email-notify-q
Content fields	subject, body (HTML or plain), attachments[]
Attachment delivery modes	inline (embedded CID image) attachment (standard MIME attachment)
Webhook provider	SendGrid delivery events at /webhooks/sendgrid
Rate limit scope	email channel in queue_controls

6.2 SMS

Attribute	Detail
Supported providers	Twilio
Queue	sms-notify-q
Content fields	body (plain text, typically ≤ 160 chars)
Recipient field	phone (E.164 format)
Webhook provider	Twilio status callbacks at /webhooks/twilio
Rate limit scope	sms channel in queue_controls

6.3 WhatsApp

Attribute	Detail
Supported providers	UltraMsg
Queue	whatsapp-notify-q
Content fields	body, optional media_url or attachment
Recipient field	wa_number (international format)
Webhook provider	UltraMsg callbacks at /webhooks/ultrams; supports batched events
Provider identifiers	provider_message_id, provider_message_sid, provider_message_hash for correlation
Rate limit scope	whatsapp channel in queue_controls

6.4 Push (FCM)

Attribute	Detail
Supported providers	Firebase Cloud Messaging (HTTP v1 API)
Queue	push-notify-q
Content fields	title, body, optional data payload
Recipient field	fcm_token (device registration token)
Authentication	Service account credentials; short-lived OAuth2 tokens cached via google-auth library
Rate limit scope	push channel in queue_controls

7 Error Handling & Troubleshooting

7.1 API Error Responses

HTTP Status	Condition	What to do
400 Bad Request	Malformed JSON, invalid date format	Fix request body; check API documentation
401 Unauthorized	Missing or invalid Bearer token	Include valid Authorization: Bearer <api_key> header
403 Forbidden	Client deactivated or insufficient permissions	Contact administrator
404 Not Found	Resource (job, provider, client) not found	Verify identifiers
409 Conflict	Duplicate provider or client	Check existing records
422 Unprocessable Entity	Validation failure; template violations	Review violations payload in response body
429 Too Many Requests	Client or global rate limit exceeded	Wait and retry; reduce request frequency
500 Internal Server Error	Unexpected server error	Retry; check server logs

7.2 Error Code Classification (DLQ)

error_code	error_type	DLQ Action
INVALID_EMAIL	PERMANENT	Drop immediately
UNSUBSCRIBED	PERMANENT	Drop immediately
AUTH_FAILED	PERMANENT	Drop immediately
RATE_LIMIT_EXCEEDED	TRANSIENT	Requeue with delay (next_retry_at or 30 min)
SMTP_TIMEOUT	TRANSIENT	Requeue with provider switch
UNKNOWN_ERROR	TRANSIENT	Requeue normally
Any (requeue_count ≥ 3)	—	Drop — max requeue attempts reached

7.3 Frequently Asked Questions

Q1. Why is my notification stuck in PROCESSING?

The worker may be paused or rate-limited. Check GET /queue-controls/{channel}. If a worker crash occurred, the job stays PROCESSING until the worker restarts (task_acks_late=True prevents re-delivery after a crash without restart).

Q2. Why was my job dropped from the DLQ?

Either the error was classified as PERMANENT (invalid address, auth failure) or the job exceeded 3 requeue attempts. Check the `drop_reason` field in the `dlq` collection.

Q3. Why did the queue switch to a backup queue?

The failover monitor detected that the primary queue depth exceeded `PRIMARY_QUEUE_CONGESTION_DEPTH` or that the queue was not draining for `PRIMARY_QUEUE_STUCK_SECONDS`. Check worker health and RabbitMQ management UI.

Q4. Why did my attachment URL expire?

Media URLs carry tokens with a TTL of `MEDIA_URL_TTL_DAYS` (default 30 days). The worker refreshes tokens automatically during retries and DLQ replays. If a URL in a stored job is expired, re-upload the file or increase `MEDIA_URL_TTL_DAYS`.

Q5. Why are push notifications not delivered?

Verify the FCM service account credentials are correctly set in provider configuration. The FCM token must be a valid, registered device token. Check that the device has not uninstalled the app or revoked notification permissions.

Q6. How do I monitor delivery rates?

Use `GET /failure-analytics` with optional `start_date` and `end_date` query parameters. The response includes grand totals and per-event-type breakdowns of sent, failed, `failure_rate`, and `delivery_rate`.

8 Configuration Reference

8.1 Environment Variables

Variable	Default	Description
<code>MONGO_URL</code>	<code>mongodb://localhost:27017</code>	MongoDB connection string
<code>CELERY_BROKER_URL</code>	<code>amqp://guest:guest@localhost:5672//</code>	RabbitMQ AMQP URL
<code>MEDIA_DIR</code>	<code>media_store/</code> (beside <code>main.py</code>)	Local directory for uploaded attachment files
<code>PUBLIC_BASE_URL</code>	(empty)	Externally reachable base URL for media link generation
<code>MEDIA_TOKEN_SECRET</code>	(empty)	Secret for HMAC-signing media access tokens
<code>MEDIA_URL_TTL_DAYS</code>	30	Days before a media token/URL expires
<code>ALLOW_PRIVATE_MEDIA_URLS</code>	0	Set to 1 to allow private/localhost source URLs
<code>ALLOWED_PRIVATE_MEDIA_HOSTS</code>	(empty)	Comma-separated hostnames treated as reachable
<code>QUEUE_FAILOVER_ENABLED</code>	1	Enable automatic queue failover monitoring
<code>PRIMARY_QUEUE_CONGESTION_DEPTH</code>	100	Messages threshold to trigger failover

Variable	Default	Description
		r
PRIMARY_QUEUE_MAX_BUFFER	500	Hard ceiling; messages beyond this are rejected
PRIMARY_QUEUE_RECOVER_DEPTH	50	Backup queue depth below which primary is restored
PRIMARY_QUEUE_STUCK_SECONDS	60	Seconds of no progress before queue is considered stuck
QUEUE_FAILOVER_MONITOR_INTERVAL_SEC	5	Failover monitor polling interval in seconds
QUEUE_FAILOVER_MIN_SWITCH_SECONDS	30	Minimum interval between consecutive queue switches

8.2 Celery Worker Start Command

Start the worker consuming all four channel queues:

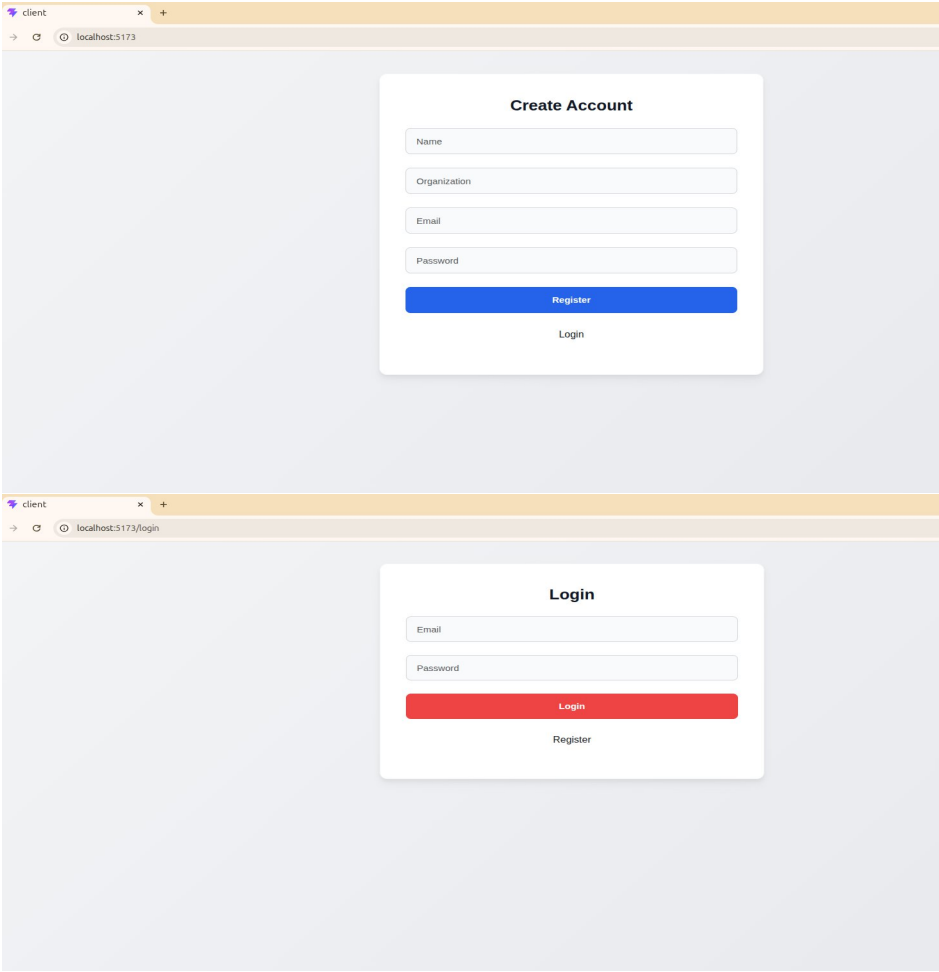
```
celery -A tasks worker --loglevel=info --concurrency=4 -Q email-notify-q,sms-notify-q,whatsapp-notify-q,push-notify-q
```

8.3 Docker Compose Services

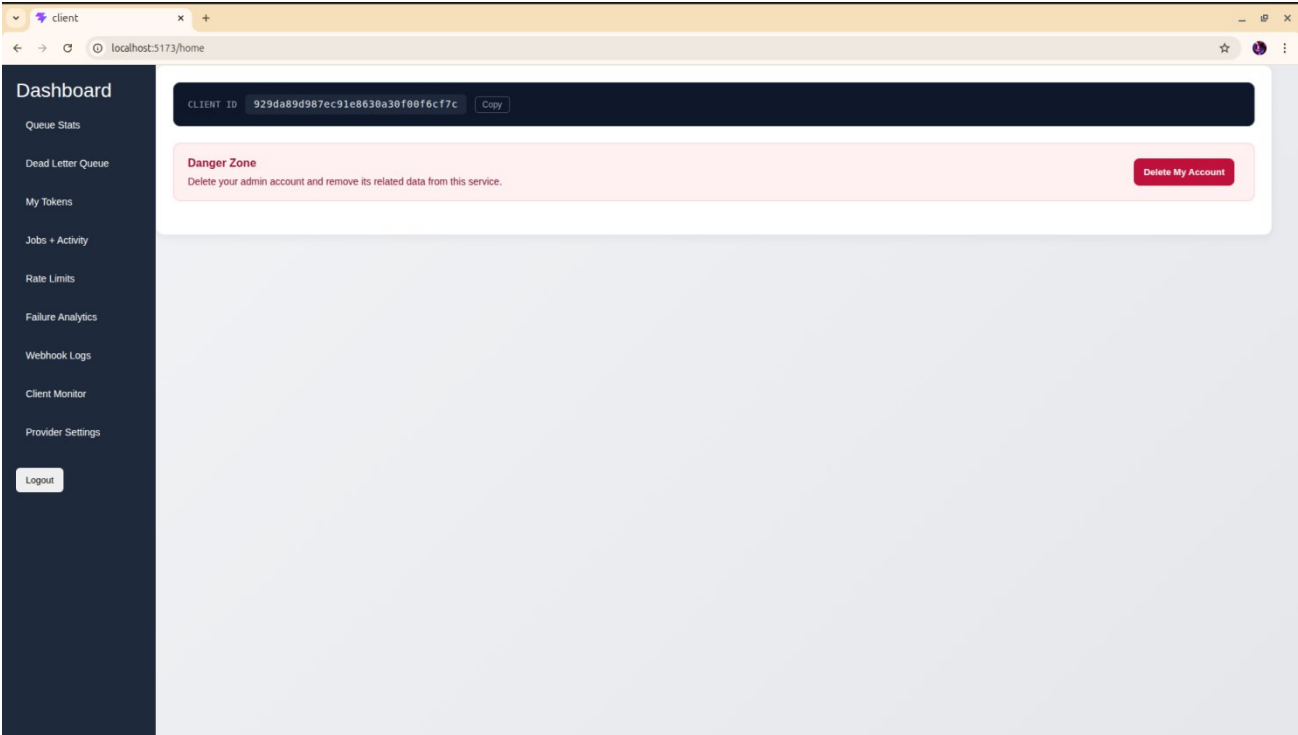
Service	Image	Port(s)	Role
mongodb	mongo:7	27017	Primary data store
rabbitmq	rabbitmq:3-management	5672, 15672	Message broker; 15672 = management UI
notification-api	python:3.11-slim	8000	FastAPI application
notification-worker	python:3.11-slim	—	Celery worker (4 concurrent tasks)
notification-client	node:20	5174	Frontend UI for the notification service
auth-server	node:20	3001	Authentication / registration service
login-client	node:20	5173	Login / registration frontend

9.Admin Dashboard pages:

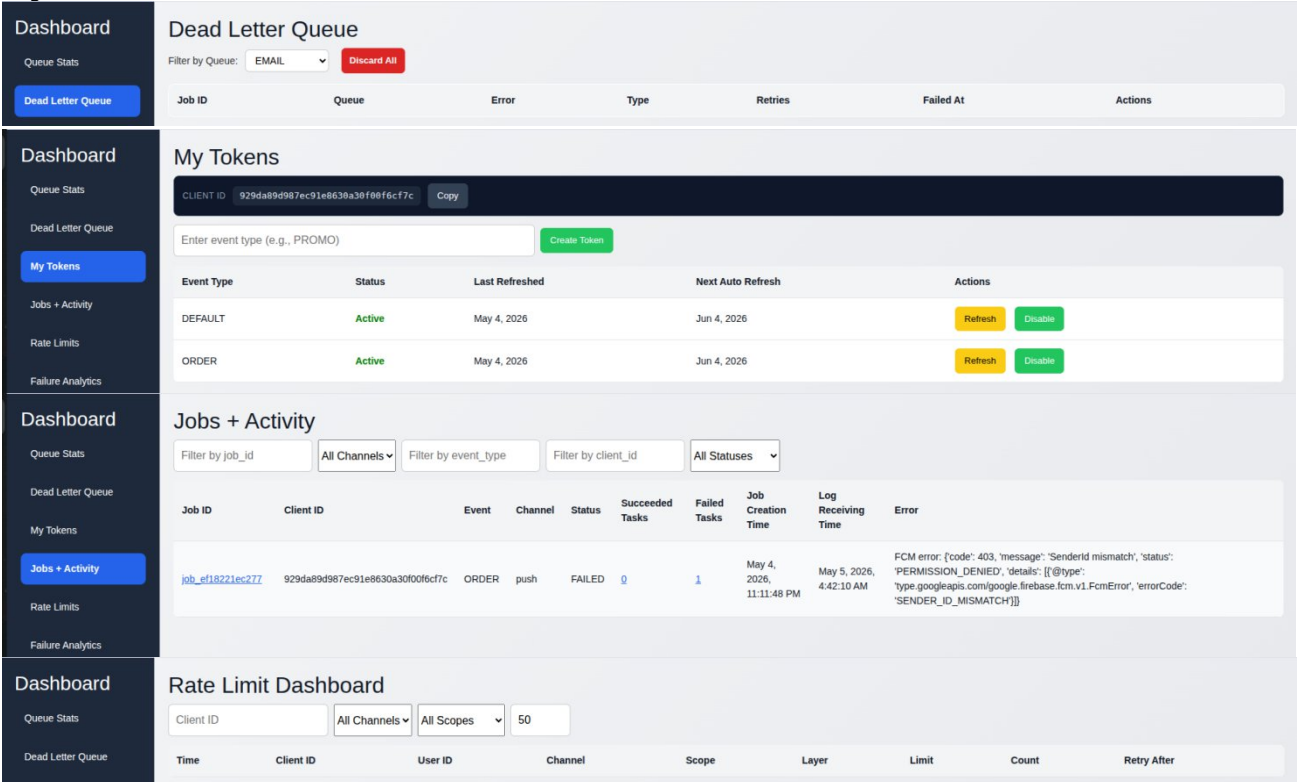
Login/ Register page:



Dashboard:



Logs:



Dashboard

Queue Stats

Dead Letter Queue

My Tokens

Failure Analytics

04/05/2026

05/05/2026

Total: 1 | Failure Rate: 100.0% | Delivery Rate: 0.0%

Event Type	Failures
ORDER	1

Dashboard

Queue Stats

Dead Letter Queue

Webhook Logs

Filter by event type

All Statuses

Log ID	Event	Status	Provider	Provider Message ID	Created At	Delivered At	Read At	Error
--------	-------	--------	----------	---------------------	------------	--------------	---------	-------